



2. Deep Neural Networks

We have many neurons. But what do exactly mean by that?

Previous Section:

[1. Neural Networks and Perceptrons](#)

Contents:

- [1. Neural Networks - The Review](#)
- [2. Deep Neural Networks - Where Layers Become Understanding](#)
- [3. The Activation Functions in Deep Neural Networks](#)
- [4. Backpropagation in Deep Neural Networks](#)
- [5. Batch, Batch Size, and Epoch in Deep Neural Networks](#)
- [6. Train a simple Deep Neural Network model](#)

[Summary](#)

[References:](#)

This section is about Deep Neural Networks. And I know that the name alone sounds like something complicated, layered, maybe even a little intimidating.

But let me write this the way I understand it. Deep Neural Networks didn't appear out of nowhere. They exist because something before them just wasn't enough. When we first study Machine Learning, everything feels structured, clean, and predictable. Data comes in tables—rows, columns, features already defined. It's like someone has already organized the world for you, and your job is simply to find patterns inside it. But that's not the only world I observe.

The data we actually live with—images, sounds, language, movement—it doesn't come labeled with neat columns. It's messy. It's raw. It doesn't tell you what matters, or even where to look.

And that's where the problem begins. Traditional Machine Learning depends on us to define meaning first. We decide what a "feature" is. We tell the model what to pay attention to. In a way, the machine is learning, but only within the boundaries we build for it.

But what if those boundaries are the limitation? What if the structure itself is something that should be discovered and not given? That question is where Deep Neural Networks begin. Instead of separating the process into "human understanding" and "machine learning," → Deep Neural Networks attempt to merge them. They take raw data as it is—and step by step, layer by layer, they build their own understanding of it.

Not because we told them what an edge is. Not because we defined what a shape looks like. But because, through enough observation, they learn to see it.

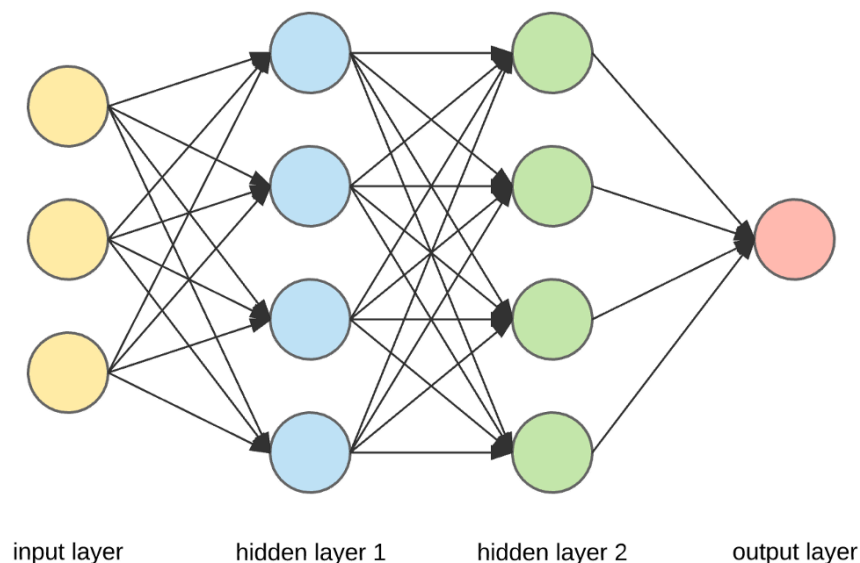
If Machine Learning is about learning from data. Then Deep Neural Networks are about learning how to understand the data itself. And at the center of that idea is something we're about to revisit carefully: Neural Networks.

1. Neural Networks - The Review

I've already written about Perceptrons before. So let's not start from zero, let's start from something familiar. A Perceptron is just **one neuron**. A Neural Network? It's simply **many neurons connected together**.

But don't let that simplicity fool you. When many simple units are connected and allowed to learn together, something much more powerful begins to emerge.

Let me break it down the way I see it (the image below is for reference only):



The BIG THREE of a Neural Network. Every Neural Network, no matter how complex it looks, is built from three core parts:

1. Input Layer — Where everything begins: This is where the data enters the system.

- Each input is a feature (usually we don't just have one value → we have many → a vector).
- Each of these inputs is assigned a **weight** → also forming a vector.
- And there is always a **bias** → another parameter that shifts the result.

So even at the very first step, the model is already asking: *"Which parts of this input matter more?"*

2. Hidden Layers — Where learning actually happens: This is the heart of the Neural Network.

- You can have **one hidden layer, or many** (this is where the “deep” in Deep Learning comes from).
- Each layer transforms the data slightly passing it forward.

And this is where all the important mechanisms live:

- **Activation Functions** → Decide how neurons “react”
- **Loss Function** → Measures how wrong the model is
- **Optimizer (via epochs & batches)** → Updates weights and bias step by step
- **Backpropagation** → The process that actually performs those updates

If the Input Layer is “receiving” → Then the Hidden Layers are **understanding**.

3. Output Layer — The final decision: After passing through all those layers, the network produces an output. Usually in the form of **probabilities**.

- Example: *Cat* → 0.5; *Dog* → 0.2; *Bird* → 0.3

So the model is not saying “this *is* a cat”. It’s saying:

| “Based on everything I’ve learned... *this is most likely a cat.*”

4. **The Bigger Picture** - So if I compress everything into one simple flow:

Input → Transform (Hidden Layers) → Predict (Output)

That’s it. That’s a Neural Network.

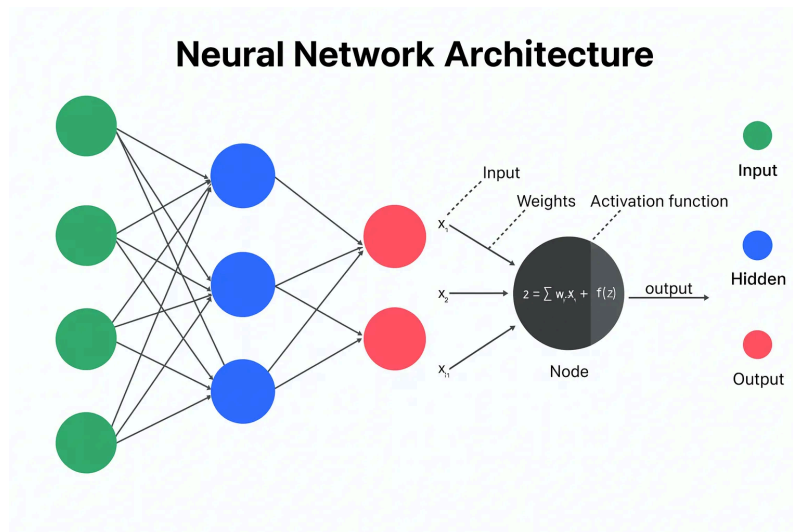
But the real beauty isn’t in the structure... It’s in the fact that **each layer learns a different level of understanding**, and together, they turn raw data into something meaningful. And once you stack many of these layers? That’s when a Neural Network becomes... **Deep**.

2. Deep Neural Networks - Where Layers Become Understanding

A Neural Network, at its core, is not complicated. It is still just neurons.

- A Perceptron is **one neuron**.
- A Neural Network is **many neurons connected together: Input → Hidden → Output**

That idea doesn’t change (reference is image below).



So what makes something a **Deep Neural Network**? Nothing new is added. No special neuron. No completely different structure. Just one thing changes:

■ ***We add more hidden layers.***

From "Shallow" to "Deep" - A standard Neural Network might look like this:

Input → Hidden → Output

But a Deep Neural Network looks like this:

Input → Hidden → Hidden → Hidden → ... → Output

Layer after layer, stacked on top of each other. And that's exactly what "deep" means. Not deep in theory Not deep in complexity (at least structurally). But literally: ***Deep = many layers***

Why Does Depth Matter? At first, adding more layers might seem unnecessary. If one layer can learn patterns, why not just use one? Because one layer can only learn **simple relationships**. But reality isn't simple.

So instead of forcing one layer to understand everything, we let multiple layers **build understanding step by step**:

- Early layers → learn simple patterns
- Middle layers → combine them into structures
- Deeper layers → recognize complex meaning

It becomes a process of transformation:

Raw data → simple signals → patterns → structures → meaning

Think of It Like This: A single neuron is like a single observation. A shallow network is like a quick interpretation. But a deep network? It's like **thinking in stages**. Each layer doesn't try to

solve the whole problem. It only solves a small part and passes it forward. And by the time the data reaches the final layer, it has been transformed multiple times—each step adding more meaning.

Deep Neural Networks are not powerful because of individual neurons. They are powerful because of **how many times the data is transformed**. Same building blocks. Same idea. Just stacked deeper. And sometimes, that depth is exactly what turns raw data into real understanding.

So what makes Deep Neural Networks stand out? At this point, it might feel like Deep Neural Networks are just bigger Neural Networks. And technically, that's true. But the reason they matter is not just because they are **deeper**— It's because that depth changes **what they are capable of learning**.

1. They can learn non-linear, complex relationships:

Simpler models (like linear models or shallow networks) are often limited to **simple patterns**. They try to draw boundaries that are straight, slightly curved. But still relatively simple.

The real world doesn't behave like that. Images, language, human behavior, all of these involve **highly complex, non-linear relationships**.

Deep Neural Networks handle this better because:

- Each layer applies transformations
- Activation functions introduce non-linearity
- Multiple layers stack these transformations together

So instead of one simple transformation, you get **many transformations composed together**, allowing the model to capture patterns that would otherwise be impossible.

2. Representation Learning (the real game changer)

This is the part that truly separates Deep Learning from traditional Machine Learning. In most Machine Learning models:

┆ **Human** → **define features** → **Model** → **learn patterns**

You decide what matters, what features to extract, and how the data should be represented.

But Deep Neural Networks flip this idea:

┆ **Raw data** → **Deep Neural Network** → **learned features + prediction**

They don't just learn patterns. They learn **how to represent the data itself**. This is called **representation learning**.

- In images → they learn edges, shapes, objects
- In text → they learn words, context, meaning
- In audio → they learn frequencies, tones, patterns

And all of this happens **without explicit human instructions**.

It's easy to misunderstand this part, so let's make it clear:

- Neural Networks (especially models like **MLP – Multi-Layer Perceptron**) are still **Machine Learning models**
- Deep Neural Networks are simply Neural Networks with many layers
- And when we use them this way, we call it **Deep Learning**

So: Deep Neural Networks = Neural Networks used in a “deep” way → powering Deep Learning

If I had to summarize what I have said so far in a simply way:

- Neural Networks → a Machine Learning model
- Deep Neural Networks → a more powerful version (many layers)
- Deep Learning → the approach that uses them to learn directly from raw data

And the reason they stand out is not just because they are deeper. It's because they remove one of the biggest limitations in Machine Learning: **The need for humans to define what matters first**. Instead, they learn that themselves.

3. The Activation Functions in Deep Neural Networks

This is where things stop being... just linear transformations.

If I had to describe Activation Functions in one sentence, I'd write it like this: They are the **hardest workers**. Because without them? A Deep Neural Network—no matter how many layers it has—would collapse into something no more powerful than a simple linear model.

What is the goal of an Activation Function? It's actually very simple → **Turn linear into non-linear**. That's it. But that “simple” step is what allows Neural Networks to:

- Learn complex patterns
- Capture real-world relationships
- Go beyond straight-line thinking

You can think of it like a switch: **Linear world** → **(Activation Function)** → **Non-linear world**

And once that switch is flipped, everything changes.

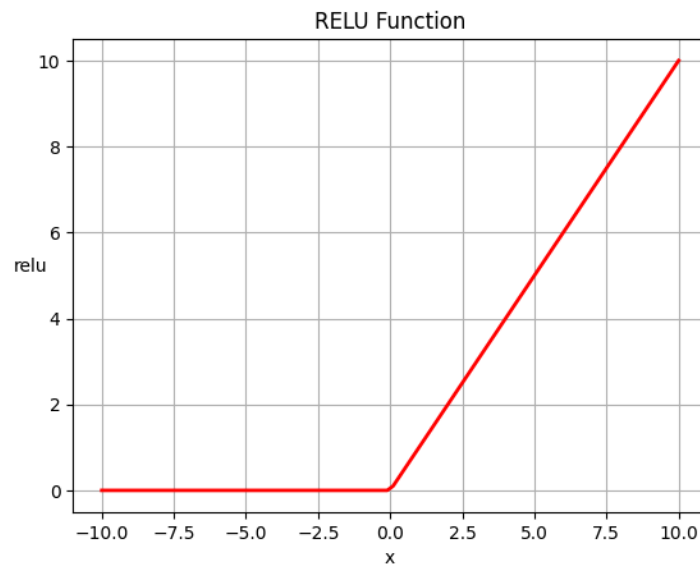
The Four Activation Functions You Must Remember: These show up everywhere. Not sometimes—**every time**.

1. ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

Output: $[0, +\infty)$ → Used in **Hidden Layers (almost always)**

Plot the function:



Why it matters:

- Solves (or at least reduces) the **vanishing gradient problem**
- Very simple → very efficient
- Allows deep networks to actually learn

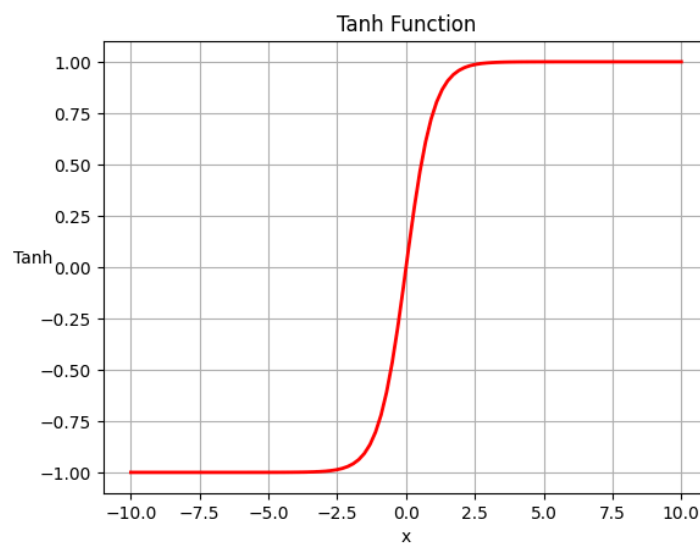
So in practice: *If you don't know what to use → use ReLU.*

2. Tanh (Hyperbolic Tangent)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output: $(-1, 1)$; Centered at 0; Used in **Hidden Layers (especially RNNs)**

Plot the function:



Why it matters:

- Keeps both **negative and positive values**
- Better than sigmoid when zero-centered output is needed

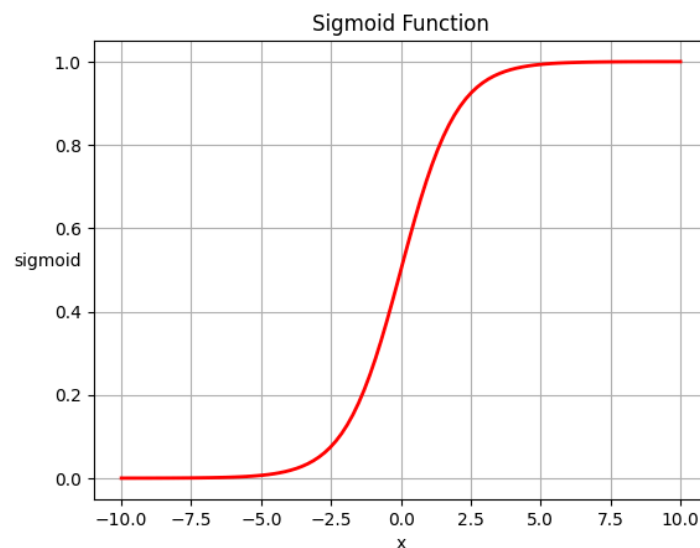
Used when the model needs **balance**, not just positivity.

3. Sigmoid (this one is familiar to most Machine Learning students)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Output: $(0, 1)$; Used in **Output Layer (Binary Classification)**

Plot the function:



Why it matters:

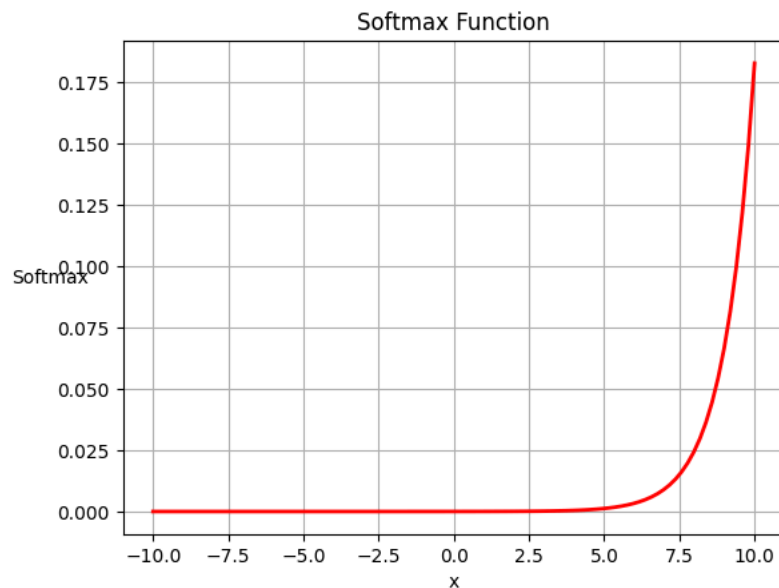
- Converts any value into a **probability**
- Very intuitive interpretation

Example: 0.9 → very likely; 0.1 → unlikely

But can suffer from **vanishing gradients** in deep layers → So we usually **don't use it in hidden layers anymore**

4. Softmax (extended from the Sigmoid function)

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$



- Output: **Probability distribution** ($sum = 1$)
- Used in **Output Layer (Multi-class classification)**

Why it matters:

- Takes multiple values → turns them into probabilities
- Ensures:
 - Each value is between 0 and 1
 - All values sum to 1

Example: *Cat* → 0.7; *Dog* → 0.2; *Bird* → 0.1

So if I compress everything into one idea:

- **ReLU / Tanh** → **Hidden Layers (learning features)**
- **Sigmoid / Softmax** → **Output Layer (making decisions)**

And the most important insight:

┃ Activation Functions are what make Deep Neural Networks *deep in capability*

Without them, stacking layers means nothing. With them? Each layer becomes a step closer to **understanding**.

4. Backpropagation in Deep Neural Networks

Now we move on to the process itself. I'll call this section *Backpropagation*. But before we actually talk about how it works, we need to understand **what is happening inside the network first**.

The Goal Never Changed - At the end of the day, a Deep Neural Network is still trying to do something very familiar: **Make a prediction.**

Just like any classification model:

- We have **inputs** (data)
- We have **outputs** (predictions as probabilities)
- And we compare them with the **actual labels**

So the Input Layer and Output Layer are connected through one purpose: Match prediction with reality.

Learning from Perceptrons - From Perceptrons, we already know the core idea:

- Start with **initial weights and bias**
- Use them to compute outputs
- Then **update them over time**

The goal? Adjust weights and bias until the prediction becomes as accurate as possible.

In other words, we want to minimize the error:

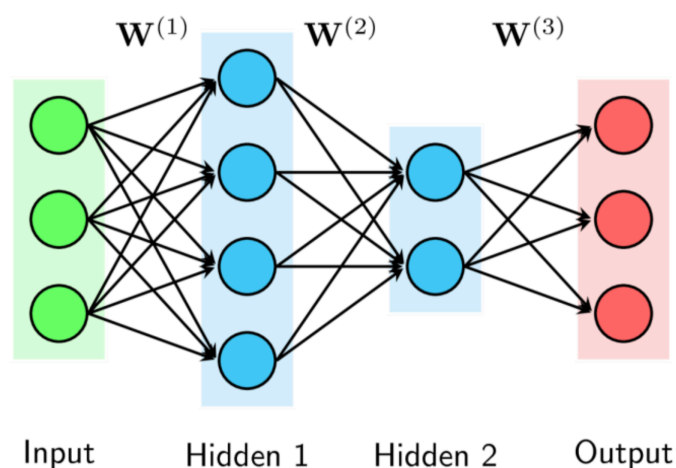
- The difference between **predicted value** and **actual value**
- Measured using a **loss function**

From One Layer... to Many Layers - In a simple case (like a single layer), the computation is straightforward: $z = w \cdot x + b$

But Deep Neural Networks are not that simple. We don't just have one transformation.

We have: $Layer \rightarrow Layer \rightarrow Layer \rightarrow \dots \rightarrow Output$

How Information Flows Through the Network - Let's imagine this step by step (You can use the figure below as visualization).



Suppose we have:

- An input layer
- Two hidden layers
- And each hidden layer has multiple neurons

Each neuron (those circles you see in diagrams) does the same thing:

- Takes **all inputs from the previous layer**
- Applies weights and bias
- Produces an output

Now here's something important: This process is **not sequential for one neuron at a time**. It is **simultaneous across all neurons in the layer**.

So what actually happens is:

- All inputs → go into **every neuron in Hidden Layer 1**
- Each neuron produces its own output
- Those outputs become inputs for **Hidden Layer 2**
- Again, all neurons process simultaneously
- And this continues until the final layer

Who Produces the Final Prediction?

At the end:

- The **last hidden layer** feeds into the **output layer**
- The output layer produces probabilities

But here's the key idea: *The final prediction is not the work of one neuron*. It is the result of every neuron → In every layer → Contributing step by step.

Before Backpropagation Begins → Once the prediction is made:

- We compare it with the **actual label**
- We compute: **Accuracy and Loss (error)**

And this is where the real question begins: If the prediction is wrong... how do we adjust all those weights across all those layers?

Because now, unlike a single Perceptron:

- We don't have just one weight
- We have **hundreds, thousands, even millions...**

And all of them contributed to the final prediction. That problem is exactly what **Backpropagation** is designed to solve.

The Backpropagation Process - Now we finally arrive at the core mechanism that makes Deep Neural Networks **actually learn**.

If I had to compress everything into one flow, it would look like this:

└ **Forward** → **Compute Loss** → **Backward** → **Update** → **Repeat**

That's it. But inside that loop... a lot is happening.

Step 1: Initialize Parameters

We start with: **Weights (W) and Biases (b)** → And they are initialized **randomly**.

At this point, the model knows nothing. Its predictions will be wrong—and that's expected.

Step 2: Forward Propagation

This is where the model makes a prediction. For each layer l :

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f\left(z^{(l)}\right)$$

- $a^{(0)} = x \rightarrow$ the input data
- $f \rightarrow$ activation function (ReLU, Tanh, etc.)

So what's happening?

- Inputs move **layer by layer**
- Each layer transforms the data
- Until we reach the output

And at the end: We get the prediction $\hat{y}$ (usually probabilities)

Step 3: Compute Loss - Now we measure how wrong the prediction is. This is done using a **loss function**.

Common choices:

- **MSE (Mean Squared Error)** = $(\hat{y} - y)$ → often for regression
- **Cross-Entropy** → more common for classification

The idea is simple: Compare predicted $\hat{y}$ with actual y → And compute: **How far off are we?**

Step 4: Backpropagation (The Core Idea)

Now comes the most important part. We need to answer:

└ *Which weights caused the error? And by how much?*

To do that, we compute gradients:

$$\frac{\partial \text{Loss}}{\partial W}, \quad \frac{\partial \text{Loss}}{\partial b}$$

Using the **chain rule**, we propagate the error:

└ **From output layer → back through hidden layers → all the way to input**

That's why it's called **Backpropagation (backward propagation of error)**

Step 5: Update Parameters (Gradient Descent)

Once we have the gradients, we update:

$$W = W - \eta \frac{\partial \text{Loss}}{\partial W}$$

$$b = b - \eta \frac{\partial \text{Loss}}{\partial b}$$

- η = **learning rate**

This step adjusts the model:

- If a weight contributed to error → reduce it
- If it helped → reinforce it

So the model becomes slightly better.

Step 6: Repeat (Batch → Epoch)

We don't do this once. We repeat it:

- For every **batch** of data
- Across many **epochs** (full passes through the dataset)

Each cycle: **Prediction → Error → Correction**

And over time: Loss decreases → Predictions improve → The model converges

The Full Loop (Final View) - Let's put everything together:

1. **Initialize W, b**
2. **Forward pass → compute $\hat{y}$**
3. **Compute loss**
4. **Backward pass → compute gradients**
5. **Update parameters**
6. **Repeat (batch → epoch)**

Backpropagation is not just a formula. It is a **systematic way of assigning blame**. Every neuron, every weight gets adjusted based on how much it contributed to the final error.

And that's how a Deep Neural Network—starting from random guesses— slowly becomes something that **understands patterns**.

5. Batch, Batch Size, and Epoch in Deep Neural Networks

We've already seen these ideas in Perceptrons. Here, nothing fundamentally changes. But now, because we are dealing with **many layers, many parameters, and large datasets**, these concepts become much more important.

The Core Idea - Training a Deep Neural Network is not: *Feed all data* → *update once* → *done*

Instead, it is a **repeated process**: *Forward* → *Loss* → *Backward* → *Update*

And we don't do this for the entire dataset at once. We break the data into smaller pieces.

What is a Batch? A **batch** is a subset of the dataset used in **one training step**.

For each batch, the model does:

Forward propagation → Compute loss → Backpropagation → Update weights and bias

So: **1 batch = 1 update**

What is Batch Size? This is the key question: *How many samples do we use before updating the model?* → That number is called the **batch size**.

Consider the input dataset: $[x_1, x_2, x_3, \dots, x_{10}]$

If **Batch Size = 2**, then:

- Batch 1 → $[x_1, x_2]$ → update
- Batch 2 → $[x_3, x_4]$ → update
- Batch 3 → $[x_5, x_6]$ → update
- Batch 4 → $[x_7, x_8]$ → update
- Batch 5 → $[x_9, x_{10}]$ → update

Number of Batches: If total samples = n and batch size = d → Then:

$$\text{Number of batches} = \frac{n}{d}$$

Example: $\frac{10}{2} = 5$ batches

What is an Epoch? An **epoch** is **one full pass through the entire dataset**.

So in the example above: **5 batches** → **completes 1 epoch**

Training Over Multiple Epochs - Obviously we don't train just once. We repeat the process:

- 5 → 10 → 30 epochs (or more, depending on the problem)

Each epoch:

- Goes through all data again
- Updates weights again
- Improves the model gradually

Why Batch Size Matters (The Key Insight) - Batch size controls **when** the model updates.

Think of it like this: You are asking students questions.

- **Batch size = 2** → ask 2 students → update your question
- Then ask the next 2 → update again

So: **Batch size controls learning frequency vs stability**

The Three Types of Gradient Descent - Based on batch size, we get three learning strategies:

1. Batch Gradient Descent (GD)

- Batch size = n (entire dataset) → Only **1 update per epoch**

Pros: Stable → Cons: Very slow (Imagine: 1 million samples → wait for all → then update → Your program might explode!!)

2. Mini-batch Gradient Descent (Most used)

- Batch size = smaller subset (e.g., 32, 64, 128) → Number of batches = $n/\text{batch size}$

Pros:

- Balance between speed and stability
- Efficient on modern hardware

This is what almost all Deep Learning models use.

3. Stochastic Gradient Descent (SGD)

- Batch size = 1 → n batches per epoch → It updates **after every single sample**.

Pros: Fast updates → Cons: Very noisy (updates jump around)

Let's compress everything:

- **Batch** → one learning step
- **Batch Size** → how much data per step
- **Epoch** → one full cycle through data

And the most important idea: Batch size is not just a number. It controls how your model *learns over time*

- Too big → slow but stable
- Too small → fast but noisy
- Just right → efficient learning

And that balance is what makes Deep Neural Networks train effectively.

6. Train a simple Deep Neural Network model

Now we bring everything together. Not theory. Not diagrams. An actual **Deep Neural Network**.

(You can run these codes in your Python interface)

Import the required library

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

Step 1: Import Data and Libraries - We'll use a very classic dataset:

```
X, y = load_iris(return_X_y=True)
```

Step 2: Prepare the Target (Very Important) - Deep Neural Networks don't work well with raw categorical labels like: 0, 1, 2 → Instead, we convert them into **One-Hot Encoding**:

```
y = keras.utils.to_categorical(y, num_classes=3)
```

With this: 0 → [1, 0, 0]; 1 → [0, 1, 0]; 2 → [0, 0, 1]. This matches perfectly with **Softmax output**.

Step 3: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
print(f"Training set shape: {X_train.shape}, {y_train.shape}")
print(f"Testing set shape: {X_test.shape}, {y_test.shape}")
```

Training set shape: (120, 4), (120, 3)

Testing set shape: (30, 4), (30, 3)

Step 4: Feature Scaling (CRITICAL for DNN) - Neural Networks are sensitive to scale.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Without this step: Training can become unstable or very slow.

Step 5: Build the Deep Neural Network - Now we define the architecture:

```
hidden_layer1 = layers.Dense(16, activation='relu')
hidden_layer2 = layers.Dense(8, activation='relu')
output_layer = layers.Dense(3, activation='softmax')
model = keras.Sequential([hidden_layer1, hidden_layer2, output_layer])
```

- **Architecture Breakdown**

- **Hidden Layer 1** → 16 neurons (ReLU)
- **Hidden Layer 2** → 8 neurons (ReLU)
- **Output Layer** → 3 neurons (Softmax)

Why?

- ReLU → standard for hidden layers
- Softmax → perfect for **multi-class classification**
- 3 outputs → because we have **3 classes**

So this is a **Deep Neural Network**: Input → Hidden 1 (16) → Hidden 2 (8) → Output (3)

Step 6: Compile the Model

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

- Let's decode this:
 - `optimizer='adam'` → A powerful version of Gradient Descent (used in Backpropagation)
 - `loss='categorical_crossentropy'` → Best for multi-class classification
 - `metrics=['accuracy']` → Measure performance

Step 7: Train the Model

```
model.fit(X_train_scaled, y_train, epochs=30, batch_size=16, validation_split=0.2)
```

- What's happening here?
 - `epochs = 30` → The model sees the data 30 times
 - `batch_size = 16` → Updates every 16 samples
 - `validation_split = 0.2` → 20% of training data used for validation

So the full loop is running: Forward → Loss → Backpropagation → Update → Repeat

- **Step 8: Evaluate the Model**

```
test_loss, test_acc = model.evaluate(X_test_scaled, y_test)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)
```

Test Loss: 0.4446692168712616

Test Accuracy: 0.8333333134651184

This tells us how well the model performs on **unseen data**, and whether it actually **learned** or just memorized

This small example already contains everything:

Data Preparation → Feature scaling → Neural Network structure → Activation functions → Forward propagation → Backpropagation → Gradient Descent → Batch, Epoch → Loss and evaluation

And that's the key realization:

*A Deep Neural Network is not complicated because of any single step. It's powerful because **all these simple steps work together**. From random weights to a model that can actually classify flowers.*

And this is just the beginning of the study of Deep Learning.

Summary

Deep Neural Networks serve as the **"first boss"** of the journey to conquer Deep Learning. Not because they are impossible to understand— But because they bring together **everything you've learned so far** into one complete system.

From Perceptrons, we took the idea of a neuron. From Neural Networks, we learned how neurons connect through layers. And with Deep Neural Networks, we pushed that idea further—stacking layers to learn **complex, non-linear relationships**.

But structure alone is not enough.

- **Activation Functions** give the network the ability to move beyond linear thinking.
- **Forward Propagation** allows the model to make predictions.

- **Loss Functions** measure how wrong those predictions are.
- **Backpropagation + Gradient Descent** adjust the system step by step.
- **Batch, Batch Size, and Epochs** control how learning happens over time.

And when all of these pieces come together. The model doesn't just learn patterns. It learns how to **represent the data itself** → That's the true power of Deep Neural Networks. They remove the need for humans to manually define features—and instead, allow the system to discover structure on its own.

They might be the “first”. Because once you understand them:

- Convolutional Neural Networks (CNNs)
- Recurrent Neural Networks (RNNs)
- Transformers

All of them are just **variations of this same idea**. Different architectures. Same foundation.

And if you can conquer this “first boss”. You're no longer just using Deep Learning. You're starting to understand it.

References:

<https://www.datacamp.com/tutorial/introduction-to-deep-neural-networks>

<https://developers.google.com/machine-learning/recommendation/dnn/softmax>

<https://www.bmc.com/blogs/deep-neural-network/>

<https://www.youtube.com/watch?v=ZBnHoVMeeJE>

Next Section:

 [3. Tensors in Deep Learning](#)